

# AI Audit Report

## 1. Thông tin nhóm

- Họ tên: Lê Trung Kiên
- MSSV: 23127075
- Nhóm: 06

## 2. Bảng audit

### 2.1. Tóm tắt audit

| STT                                                                                                                  | Prompt + Tool                                                | Verdict |
|----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|---------|
| 1                                                                                                                    | Time:2026-08-31 13:46<br>+07Tool: Codex / GPT-5Prompt--<br>- |         |
| name: writing-skills                                                                                                 |                                                              |         |
| description: Use when creating new skills, editing<br>existing skills, or verifying skills work before<br>deployment |                                                              |         |

# Writing Skills

## Overview

Writing skills IS Test-Driven Development applied to process documentation.

Personal skills live in your runtime's skills directory (~/.claude/skills/ on Claude Code) — see [codex-tools.md](#) or [gemini-tools.md](#) for the path on those runtimes. Codex, Copilot CLI, and Gemini CLI all also recognize ~/.agents/skills/ as a cross-runtime alias.

You write test cases (pressure scenarios with subagents), watch them fail (baseline behavior), write the skill (documentation), watch tests pass (agents comply), and refactor (close loopholes).

**Core principle:** If you didn't watch an agent fail without the skill, you don't know if the skill teaches the right thing.

**REQUIRED BACKGROUND:** You MUST understand superpowers:test-driven-development before using this skill. That skill defines the fundamental RED-GREEN-REFACTOR cycle. This skill adapts TDD to documentation.

**Official guidance:** For Anthropic's official skill authoring best practices, see anthropic-best-practices.md. This document provides additional patterns and guidelines that complement the TDD-focused approach in this skill.

# What is a Skill?

A **skill** is a reference guide for proven techniques, patterns, or tools. Skills help future agents find and apply effective approaches.

**Skills are:** Reusable techniques, patterns, tools, reference guides

**Skills are NOT:** Narratives about how you solved a problem once

## TDD Mapping for Skills

| TDD Concept         | Skill Creation                                   |
|---------------------|--------------------------------------------------|
| Test case           | Pressure scenario with subagent                  |
| Production code     | Skill document (SKILL.md)                        |
| Test fails (RED)    | Agent violates rule without skill (baseline)     |
| Test passes (GREEN) | Agent complies with skill present                |
| Refactor            | Close loopholes while maintaining compliance     |
| Write test first    | Run baseline scenario BEFORE writing skill       |
| Watch it fail       | Document exact rationalizations agent uses       |
| Minimal code        | Write skill addressing those specific violations |
| Watch it pass       | Verify agent now complies                        |
| Refactor cycle      | Find new rationalizations → plug → re-verify     |

The entire skill creation process follows RED-GREEN-REFACTOR.

## When to Create a Skill

### Create when:

- Technique wasn't intuitively obvious to you
- You'd reference this again across projects
- Pattern applies broadly (not project-specific)
- Others would benefit

### Don't create for:

- One-off solutions
- Standard practices well-documented elsewhere
- Project-specific conventions (put in your instructions file)
- Mechanical constraints (if it's enforceable with regex/validation, automate it—save documentation for judgment calls)

## Skill Types

### Technique

Concrete method with steps to follow (condition-based-waiting, root-cause-tracing)

## Pattern

Way of thinking about problems (flatten-with-flags, test-invariants)

## Reference

API docs, syntax guides, tool documentation (office docs)

## Directory Structure

```
skills/  
  skill-name/  
    SKILL.md           # Main reference (required)  
    supporting-file.*  # Only if needed
```

**Flat namespace** - all skills in one searchable namespace

### Separate files for:

1. **Heavy reference** (100+ lines) - API docs, comprehensive syntax
2. **Reusable tools** - Scripts, utilities, templates

### Keep inline:

- Principles and concepts
- Code patterns (< 50 lines)
- Everything else

## SKILL.md Structure

### Frontmatter (YAML):

- Two required fields: **name** and **description** (see [agentskills.io/specification](https://agentskills.io/specification) for all supported fields)
- Max 1024 characters total
- **name**: Use letters, numbers, and hyphens only (no parentheses, special chars)
- **description**: Third-person, describes ONLY when to use (NOT what it does)
  - Start with "Use when..." to focus on triggering conditions
  - Include specific symptoms, situations, and contexts
  - **NEVER summarize the skill's process or workflow** (see SDO section for why)
  - Keep under 500 characters if possible

```
---  
name: Skill-Name-With-Hyphens  
description: Use when [specific triggering conditions and symptoms]  
---  
  
# Skill Name
```

## ## Overview

What is this? Core principle in 1-2 sentences.

## ## When to Use

[Small inline flowchart IF decision non-obvious]

Bullet list with SYMPTOMS and use cases

When NOT to use

## ## Core Pattern (for techniques/patterns)

Before/after code comparison

## ## Quick Reference

Table or bullets for scanning common operations

## ## Implementation

Inline code for simple patterns

Link to file for heavy reference or reusable tools

## ## Common Mistakes

What goes wrong + fixes

## ## Real-World Impact (optional)

Concrete results

# Skill Discovery Optimization (SDO)

**Critical for discovery:** Future agents need to FIND your skill

## 1. Rich Description Field

**Purpose:** Your agent reads the description to decide which skills to load for a given task. Make it answer: "Should I read this skill right now?"

**Format:** Start with "Use when..." to focus on triggering conditions

**CRITICAL: Description = When to Use, NOT What the Skill Does**

The description should ONLY describe triggering conditions. Do NOT summarize the skill's process or workflow in the description.

**Why this matters:** Testing revealed that when a description summarizes the skill's workflow, an agent may follow the description instead of reading the full skill content. A description saying "code review between tasks" caused an agent to do ONE review, even though the skill's flowchart clearly showed TWO reviews (spec compliance then code quality).

When the description was changed to just "Use when executing implementation plans with independent tasks" (no workflow summary), the agent correctly read the flowchart and followed the two-stage review process.

**The trap:** Descriptions that summarize workflow create a shortcut agents will take. The skill body becomes documentation agents skip.

```
# ❌ BAD: Summarizes workflow - agents may follow this instead of reading skill
description: Use when executing plans - dispatches subagent per task with code review between tasks

# ❌ BAD: Too much process detail
description: Use for TDD - write test first, watch it fail, write minimal code, refactor

# ✅ GOOD: Just triggering conditions, no workflow summary
description: Use when executing implementation plans with independent tasks in the current session

# ✅ GOOD: Triggering conditions only
description: Use when implementing any feature or bugfix, before writing implementation code
```

### Content:

- Use concrete triggers, symptoms, and situations that signal this skill applies
- Describe the *problem* (race conditions, inconsistent behavior) not *language-specific symptoms* (setTimeout, sleep)
- Keep triggers technology-agnostic unless the skill itself is technology-specific
- If skill is technology-specific, make that explicit in the trigger
- Write in third person (injected into system prompt)
- **NEVER summarize the skill's process or workflow**

```
# ❌ BAD: Too abstract, vague, doesn't include when to use
description: For async testing

# ❌ BAD: First person
description: I can help you with async tests when they're flaky

# ❌ BAD: Mentions technology but skill isn't specific to it
description: Use when tests use setTimeout/sleep and are flaky

# ✅ GOOD: Starts with "Use when", describes problem, no workflow
description: Use when tests have race conditions, timing dependencies, or pass/fail inconsistently

# ✅ GOOD: Technology-specific skill with explicit trigger
description: Use when using React Router and handling authentication redirects
```

## 2. Keyword Coverage

Use words an agent would search for:

- Error messages: "Hook timed out", "ENOTEMPTY", "race condition"
- Symptoms: "flaky", "hanging", "zombie", "pollution"
- Synonyms: "timeout/hang/freeze", "cleanup/teardown/afterEach"
- Tools: Actual commands, library names, file types

### 3. Descriptive Naming

**Use active voice, verb-first:**

-  **creating-skills** not **skill-creation**
-  **condition-based-waiting** not **async-test-helpers**

### 4. Token Efficiency (Critical)

**Problem:** getting-started and frequently-referenced skills load into EVERY conversation. Every token counts.

**Target word counts:**

- getting-started workflows: <150 words each
- Frequently-loaded skills: <200 words total
- Other skills: <500 words (still be concise)

**Techniques:**

**Move details to tool help:**

```
#  BAD: Document all flags in SKILL.md
search-conversations supports --text, --both, --after DATE, --before DATE,
--limit N

#  GOOD: Reference --help
search-conversations supports multiple modes and filters. Run --help for
details.
```

**Use cross-references:**

```
#  BAD: Repeat workflow details
When searching, dispatch subagent with template...
[20 lines of repeated instructions]

#  GOOD: Reference other skill
Always use subagents (50-100x context savings). REQUIRED: Use [other-skill-
name] for workflow.
```

**Compress examples:**

```
# ❌ BAD: Verbose example (42 words)
your human partner: "How did we handle authentication errors in React Router before?"
You: I'll search past conversations for React Router authentication patterns.
[Dispatch subagent with search query: "React Router authentication error handling 401"]

# ✅ GOOD: Minimal example (20 words)
Partner: "How did we handle auth errors in React Router?"
You: Searching...
[Dispatch subagent → synthesis]
```

### Eliminate redundancy:

- Don't repeat what's in cross-referenced skills
- Don't explain what's obvious from command
- Don't include multiple examples of same pattern

### Verification:

```
WC -w skills/path/SKILL.md
# getting-started workflows: aim for <150 each
# Other frequently-loaded: aim for <200 total
```

### Name by what you DO or core insight:

- ✅ condition-based-waiting > async-test-helpers
- ✅ using-skills not skill-usage
- ✅ flatten-with-flags > data-structure-refactoring
- ✅ root-cause-tracing > debugging-techniques

### Gerunds (-ing) work well for processes:

- creating-skills, testing-skills, debugging-with-logs
- Active, describes the action you're taking

## 5. Cross-Referencing Other Skills

### When writing documentation that references other skills:

Use skill name only, with explicit requirement markers:

- ✅ Good: **\*\*REQUIRED SUB-SKILL:\*\*** Use superpowers:test-driven-development
- ✅ Good: **\*\*REQUIRED BACKGROUND:\*\*** You MUST understand superpowers:systematic-debugging
- ❌ Bad: See skills/testing/test-driven-development (unclear if required)
- ❌ Bad: @skills/testing/test-driven-development/SKILL.md (force-loads, burns context)

**Why no @ links:** @ syntax force-loads files immediately, consuming 200k+ context before you need them.

## Flowchart Usage

```
digraph when_flowchart {
    "Need to show information?" [shape=diamond];
    "Decision where I might go wrong?" [shape=diamond];
    "Use markdown" [shape=box];
    "Small inline flowchart" [shape=box];

    "Need to show information?" -> "Decision where I might go wrong?"
    [label="yes"];
    "Decision where I might go wrong?" -> "Small inline flowchart"
    [label="yes"];
    "Decision where I might go wrong?" -> "Use markdown" [label="no"];
}
```

### Use flowcharts ONLY for:

- Non-obvious decision points
- Process loops where you might stop too early
- "When to use A vs B" decisions

### Never use flowcharts for:

- Reference material → Tables, lists
- Code examples → Markdown blocks
- Linear instructions → Numbered lists
- Labels without semantic meaning (step1, helper2)

See [graphviz-conventions.dot](#) in this directory for graphviz style rules.

**Visualizing for your human partner:** Use [render-graphs.js](#) in this directory to render a skill's flowcharts to SVG:

```
./render-graphs.js ../some-skill # Each diagram separately
./render-graphs.js ../some-skill --combine # All diagrams in one SVG
```

## Code Examples

### One excellent example beats many mediocre ones

Choose most relevant language:

- Testing techniques → TypeScript/JavaScript
- System debugging → Shell/Python
- Data processing → Python



**Good example:**

- Complete and runnable
- Well-commented explaining WHY
- From real scenario
- Shows pattern clearly
- Ready to adapt (not generic template)

**Don't:**

- Implement in 5+ languages
- Create fill-in-the-blank templates
- Write contrived examples

You're good at porting - one great example is enough.

## File Organization

### Self-Contained Skill

```
defense-in-depth/  
SKILL.md      # Everything inline
```

When: All content fits, no heavy reference needed

### Skill with Reusable Tool

```
condition-based-waiting/  
SKILL.md      # Overview + patterns  
example.ts    # Working helpers to adapt
```

When: Tool is reusable code, not just narrative

### Skill with Heavy Reference

```
pptx/  
SKILL.md      # Overview + workflows  
pptxgenjs.md  # 600 lines API reference  
ooxml.md      # 500 lines XML structure  
scripts/      # Executable tools
```

When: Reference material too large for inline

## The Iron Law (Same as TDD)

NO SKILL WITHOUT A FAILING TEST FIRST

This applies to NEW skills AND EDITS to existing skills.

Write skill before testing? Delete it. Start over. Edit skill without testing? Same violation.

**No exceptions:**

- Not for "simple additions"
- Not for "just adding a section"
- Not for "documentation updates"
- Don't keep untested changes as "reference"
- Don't "adapt" while running tests
- Delete means delete

**REQUIRED BACKGROUND:** The superpowers:test-driven-development skill explains why this matters. Same principles apply to documentation.

## Testing All Skill Types

Different skill types need different test approaches:

### Discipline-Enforcing Skills (rules/requirements)

**Examples:** TDD, verification-before-completion, designing-before-coding

**Test with:**

- Academic questions: Do they understand the rules?
- Pressure scenarios: Do they comply under stress?
- Multiple pressures combined: time + sunk cost + exhaustion
- Identify rationalizations and add explicit counters

**Success criteria:** Agent follows rule under maximum pressure

### Technique Skills (how-to guides)

**Examples:** condition-based-waiting, root-cause-tracing, defensive-programming

**Test with:**

- Application scenarios: Can they apply the technique correctly?
- Variation scenarios: Do they handle edge cases?
- Missing information tests: Do instructions have gaps?

**Success criteria:** Agent successfully applies technique to new scenario

### Pattern Skills (mental models)

**Examples:** reducing-complexity, information-hiding concepts

Test with:

- Recognition scenarios: Do they recognize when pattern applies?
- Application scenarios: Can they use the mental model?
- Counter-examples: Do they know when NOT to apply?

**Success criteria:** Agent correctly identifies when/how to apply pattern

Reference Skills (documentation/APIs)

**Examples:** API documentation, command references, library guides

Test with:

- Retrieval scenarios: Can they find the right information?
- Application scenarios: Can they use what they found correctly?
- Gap testing: Are common use cases covered?

**Success criteria:** Agent finds and correctly applies reference information

Common Rationalizations for Skipping Testing

| Excuse                         | Reality                                                          |
|--------------------------------|------------------------------------------------------------------|
| "Skill is obviously clear"     | Clear to you ≠ clear to other agents. Test it.                   |
| "It's just a reference"        | References can have gaps, unclear sections. Test retrieval.      |
| "Testing is overkill"          | Untested skills have issues. Always. 15 min testing saves hours. |
| "I'll test if problems emerge" | Problems = agents can't use skill. Test BEFORE deploying.        |
| "Too tedious to test"          | Testing is less tedious than debugging bad skill in production.  |
| "I'm confident it's good"      | Overconfidence guarantees issues. Test anyway.                   |
| "Academic review is enough"    | Reading ≠ using. Test application scenarios.                     |
| "No time to test"              | Deploying untested skill wastes more time fixing it later.       |

**All of these mean: Test before deploying. No exceptions.**

Match the Form to the Failure

Before writing guidance, classify the baseline failure. The form that bulletproofs one failure type measurably backfires on another.

| Baseline failure                                                    | Right form                                                                 | Wrong form                                 |
|---------------------------------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------|
| Skips/violates a rule under pressure (knows better, does it anyway) | Prohibition + rationalization table + red flags (see Bulletproofing below) | Soft guidance ("prefer...", "consider...") |
| Complies, but output has the wrong shape (bloated prompt, buried    | Positive recipe or contract: state what the output IS — its parts, in      | Prohibition list ("don't restate", "never  |

| Baseline failure                                             | Right form                                                                         | Wrong form                             |
|--------------------------------------------------------------|------------------------------------------------------------------------------------|----------------------------------------|
| verdict, restated spec)                                      | order                                                                              | narrate")                              |
| Omits a required element from something they already produce | Structural: REQUIRED field or slot in the template they fill in                    | Prose reminders near the template      |
| Behavior should depend on a condition                        | Conditional keyed to an observable predicate ("if the brief exists, reference it") | Unconditional rule + exemption clauses |

**Why prohibitions backfire on shaping problems:** under a competing incentive ("make the prompt self-contained"), agents negotiate with "don't X". In head-to-head wording tests on dispatch-prompt guidance, the prohibition arm produced clearly more of the unwanted content than the recipe arm (fully separated distributions), and trended worse than even the no-guidance control — micro-test your own case rather than assuming, but never reach for the prohibition by default. A recipe leaves nothing to negotiate: the output matches the stated shape or it doesn't.

**Rules for whichever form you pick:**

- **No nuance clauses.** "Don't X unless it matters" reopens the negotiation — appending a single nuance clause to a winning recipe degraded it from consistent to noisy in the same wording tests. Express a real exception as its own conditional on an observable predicate.
- **Exemption clauses don't scope.** "This limit doesn't apply to code blocks" still suppresses code blocks. If part of the output must be exempt, restructure so the rule can't reach it.

## Bulletproofing Skills Against Rationalization

Skills that enforce discipline (like TDD) need to resist rationalization. Agents are smart and will find loopholes when under pressure.

**Scope:** this toolkit is for discipline failures — an agent that knows the rule and skips it under pressure. For wrong-shaped output or omitted elements, prohibition-based bulletproofing backfires; use the forms in Match the Form to the Failure instead.

**Psychology note:** Understanding WHY persuasion techniques work helps you apply them systematically. See persuasion-principles.md for research foundation (Cialdini, 2021; Meincke et al., 2025) on authority, commitment, scarcity, social proof, and unity principles.

### Close Every Loophole Explicitly

Don't just state the rule - forbid specific workarounds:

`` `markdown Write code before test? Delete it. `` ` ` `markdown Write code before test? Delete it. Start over.

**No exceptions:**

- Don't keep it as "reference"
- Don't "adapt" it while writing tests
- Don't look at it

- Delete means delete

</Good>

### Address "Spirit vs Letter" Arguments

Add foundational principle early:

```markdown

**\*\*Violating the letter of the rules is violating the spirit of the rules.\*\***

This cuts off entire class of "I'm following the spirit" rationalizations.

## Build Rationalization Table

Capture rationalizations from baseline testing (see Testing section below). Every excuse agents make goes in the table:

```
| Excuse | Reality |
|-----|-----|
| "Too simple to test" | Simple code breaks. Test takes 30 seconds. |
| "I'll test after" | Tests passing immediately prove nothing. |
| "Tests after achieve same goals" | Tests-after = "what does this do?"
Tests-first = "what should this do?" |
```

## Create Red Flags List

Make it easy for agents to self-check when rationalizing:

**## Red Flags - STOP and Start Over**

- Code before test
- "I already manually tested it"
- "Tests after achieve the same purpose"
- "It's about spirit not ritual"
- "This is different because..."

**\*\*All of these mean: Delete code. Start over with TDD.\*\***

## Update SDO for Violation Symptoms

Add to description: symptoms of when you're ABOUT to violate the rule:

description: use when implementing any feature or bugfix, before writing implementation code

## RED-GREEN-REFACTOR for Skills

Follow the TDD cycle:

### RED: Write Failing Test (Baseline)

Run pressure scenario with subagent WITHOUT the skill. Document exact behavior:

- What choices did they make?
- What rationalizations did they use (verbatim)?
- Which pressures triggered violations?

This is "watch the test fail" - you must see what agents naturally do before writing the skill.

### GREEN: Write Minimal Skill

Write skill that addresses those specific rationalizations. Don't add extra content for hypothetical cases.

Run same scenarios WITH skill. Agent should now comply.

### REFACTOR: Close Loopholes

Agent found new rationalization? Add explicit counter. Re-test until bulletproof.

### Micro-Test Wording Before Full Scenarios

Full pressure-scenario runs are the final gate, but they are slow and expensive per iteration. Verify the wording itself first with micro-tests:

1. **One fresh-context sample per call** — a raw API call, or a single-shot subagent if you don't have API access. System prompt = the realistic context the guidance will live in (the full skill or prompt template, not the guidance in isolation); user message = a task that tempts the failure.
2. **Always include a no-guidance control.** If the control doesn't exhibit the failure, there is nothing to fix — stop, don't author the guidance.
3. **5+ reps per variant.** Single samples lie.
4. **Manually read every flagged match.** Score programmatically if you like, but template echoes and quoted counter-examples masquerade as hits; automated counts alone overstate both failure and success.
5. **Variance is a metric.** When guidance lands, reps converge on the same shape. Five different interpretations across five reps means the wording isn't binding — tighten the form before adding words.

Micro-tests verify wording; they do not replace pressure scenarios for discipline skills.

**Testing methodology:** See [testing-skills-with-subagents.md](#) for the complete testing methodology:

- How to write pressure scenarios
- Pressure types (time, sunk cost, authority, exhaustion)
- Plugging holes systematically
- Meta-testing techniques

## Anti-Patterns

### ✗ Narrative Example

"In session 2025-10-03, we found empty projectDir caused..." **Why bad:** Too specific, not reusable

### ✗ Multi-Language Dilution

example-js.js, example-py.py, example-go.go **Why bad:** Mediocre quality, maintenance burden

### ✗ Code in Flowcharts

```
step1 [label="import fs"];  
step2 [label="read file"];
```

**Why bad:** Can't copy-paste, hard to read

### ✗ Generic Labels

helper1, helper2, step3, pattern4 **Why bad:** Labels should have semantic meaning

## STOP: Before Moving to Next Skill

**After writing ANY skill, you MUST STOP and complete the deployment process.**

### Do NOT:

- Create multiple skills in batch without testing each
- Move to next skill before current one is verified
- Skip testing because "batching is more efficient"

**The deployment checklist below is MANDATORY for EACH skill.**

Deploying untested skills = deploying untested code. It's a violation of quality standards.

## Skill Creation Checklist (TDD Adapted)

**IMPORTANT: Create a todo for EACH checklist item below.**

### RED Phase - Write Failing Test:

- ☐ Create pressure scenarios (3+ combined pressures for discipline skills)
- ☐ Run scenarios WITHOUT skill - document baseline behavior verbatim
- ☐ Identify patterns in rationalizations/failures

**GREEN Phase - Write Minimal Skill:**

- ☐ Name uses only letters, numbers, hyphens (no parentheses/special chars)
- ☐ YAML frontmatter with required **name** and **description** fields (max 1024 chars; see [spec](#))
- ☐ Description starts with "Use when..." and includes specific triggers/symptoms
- ☐ Description written in third person
- ☐ Keywords throughout for search (errors, symptoms, tools)
- ☐ Clear overview with core principle
- ☐ Address specific baseline failures identified in RED
- ☐ Guidance form matches the failure type (see Match the Form to the Failure)
- ☐ For behavior-shaping guidance: wording micro-tested against a no-guidance control (5+ reps, every flagged match read manually) — N/A for pure reference skills
- ☐ Code inline OR link to separate file
- ☐ One excellent example (not multi-language)
- ☐ Run scenarios WITH skill - verify agents now comply

**REFACTOR Phase - Close Loopholes:**

- ☐ Identify NEW rationalizations from testing
- ☐ Add explicit counters (if discipline skill)
- ☐ Build rationalization table from all test iterations
- ☐ Create red flags list
- ☐ Re-test until bulletproof

**Quality Checks:**

- ☐ Small flowchart only if decision non-obvious
- ☐ Quick reference table
- ☐ Common mistakes section
- ☐ No narrative storytelling
- ☐ Supporting files only for tools or heavy reference

**Deployment:**

- ☐ Commit skill to git and push to your fork (if configured)
- ☐ Consider contributing back via PR (if broadly useful)

## Discovery Workflow

How future agents find your skill:

1. **Encounters problem** ("tests are flaky")
2. **Searches skills** (greps descriptions, browses categories)
3. **Finds SKILL** (description matches)
4. **Scans overview** (is this relevant?)
5. **Reads patterns** (quick reference table)
6. **Loads example** (only when implementing)

**Optimize for this flow** - put searchable terms early and often. Cập nhật [\\$ai-audit-entry](#) để phù hợp với project hiện tại. Folder src chính là folder cho bài làm. Sau đó sẽ copy ra thành folder để zip lại, đổi tên rồi



nộp. Nhớ cập nhật cả AGENTS.md | Passed with Modification | | 2 | Time:2026-09-01 00:30 +07Tool: Gemini 3.6 FlashPrompt:/using-superpowers Hãy cài đặt những công cụ / thư viện... cần thiết phục vụ cho homework này. Viết audit | Accepted | | 3 | Time:2026-09-01 16:08 +07Tool: Gemini 3.6 FlashPrompt:/using-superpowers Dựa vào folder req (chứa đề bài tập) và các spec trong folder eshop-sut (eshop-sut/README.md và eshop-sut/api\_specification.md). Hãy tạo 2 file CSV dữ liệu đầu vào cho luồng Admin (Member 4) tại src/data/credentials.csv (chứa tài khoản admin@eshop.com) và src/data/products.csv (chứa dữ liệu sản phẩm mẫu). Lưu ý có thể tham khảo mã nguồn của eshop-sut để đảm bảo data tạo ra có thể dùng được. Đưa ghi nhận vào AI Audit Report. | Accepted | | 4 | Time:2026-09-01 16:14 +07Tool: Claude Opus 4.6Prompt:/subagent-driven-development Tôi đã tạo data tại src/data. Dựa vào yêu cầu trong folder req, hãy thiết kế kịch bản Load Test Plan cho luồng Admin của sinh viên 23127075 (10 threads, ramp-up 10s, dùng Aggregate Report) cho project eshop-sut và lưu tại src/test-plans/23127075\_Load\_20260901.jmx. Bạn có thể sửa test data nếu chưa phù hợp. Đưa ghi nhận vào AI Audit Report. | Passed with Modification | | 5 | Time:2026-09-01 16:19 +07Tool: Claude Opus 4.6Prompt:/using-superpowers Tôi đã tạo data tại src/data. Dựa vào yêu cầu trong folder req, hãy tạo kịch bản Stress Test Plan cho luồng Admin (50 threads, ramp-up 15s, dùng Summary Report listener) cho project eshop-sut và lưu tại src/test-plans/23127075\_Stress\_20260901.jmx. Bạn có thể sửa test data nếu chưa phù hợp. Đưa ghi nhận vào AI Audit Report. | Passed with Modification | | 6 | Time:2026-09-01 16:24 +07Tool: Claude Opus 4.6Prompt:/using-superpowers Tôi đã tạo data tại src/data. Dựa vào yêu cầu trong folder req, hãy tạo kịch bản Spike Test Plan cho luồng Admin (100 threads, ramp-up 1s, dùng View Results Tree listener) cho project eshop-sut và lưu tại src/test-plans/23127075\_Spike\_20260901.jmx. Bạn có thể sửa test data nếu chưa phù hợp. Đưa ghi nhận vào AI Audit Report. | Passed with Modification | | 7 | Time:2026-09-01 19:28 +07Tool: Gemini 3.6 FlashPrompt:/using-superpowers Do thư mục src sẽ được copy ra thành thư mục nộp độc lập, hãy tạo script tự động chạy kiểm thử tại src/run\_tests.sh và cập nhật lại toàn bộ đường dẫn tương đối trong các kịch bản JMeter (.jmx) cho phù hợp với thư mục thực thi src. Đưa ghi nhận vào AI Audit Report. | Passed with Modification | | 8 | Time:2026-09-01 20:17 +07Tool: Gemini 3.6 FlashPrompt:/using-superpowers Hãy convert .agents/skills/ai-audit-entry/ref/13\_Performance Testing.pdf ra markdown đầy đủ nội dung, layout (cố gắng convert full cả table, diagram thay vì cap ảnh) | Passed with Modification | | 9 | Time:2026-09-01 21:23 +07Tool: Claude Opus 4.6Prompt:/using-superpowers /writing-skills Hãy viết agent skill: performance-testing-skills để automate hoàn thành bài tập theo đề trong folder req với các yêu cầu sau:- Tham khảo lý thuyết từ slide (file markdown) trong folder ref.- Tham khảo src/ai-audit/ai\_audit\_report.md để hiểu quá trình làm bài.- Bộ skill phải đầy đủ bước cho cả workflow và đảm bảo tính độc lập, tái sử dụng, portable, không lệ thuộc vào các resource (như kiểu phải đưa slide vào để học kiến thức).- Output skill nằm trong .agents/skillsHãy viết 1 file hướng dẫn sử dụng bộ skill này ở root. Ghi prompt này vào ai-audit. | Passed with Modification | | 10 | Time:2026-09-03 00:24 +07Tool: Codex / GPT-5Prompt:/using-superpowers /performance-testing-skills Hãy rà soát bài làm so với đề, sửa JMX để kết quả có thể tổng hợp ổn định, tạo Endurance 10 phút có log CPU/RSS thật, thêm công cụ phân tích JTL và validator đối chiếu JTL/HTML, sau đó cập nhật báo cáo bằng số liệu đã chạy. Không tạo PDF, video hoặc bằng chứng GitHub giả. | Passed with Modification |

## 2.2. Chi tiết audit

### 2.2.1 Entry 1

#### Prompt + Tool:

Time: 2026-08-31 13:46 +07 Tool: Codex / GPT-5 Prompt: \$superpowers:writing-skills Cập nhật \$ai-audit-entry để phù hợp với project hiện tại. Folder src chính là folder cho bài làm. Sau đó sẽ copy ra thành

folder để zip lại, đổi tên rồi nộp. Nhớ cập nhật cả AGENTS.md

**AI Output:** Đã cập nhật `.agents/skills/ai-audit-entry/SKILL.md`, script audit, và `AGENTS.md` để dùng `src/` làm thư mục làm việc; đã kiểm tra smoke test và diff.

**Verdict:** Passed with Modification

**Reasoning:** Dựa trên nguyên tắc Adaptability & Human Oversight (ISTQB CT-AI 2.1 & 2.2), AI đề xuất cấu trúc lưu nhật ký audit dưới thư mục wrapper `src/submission/ai-audit/`. Điều này giúp phân tách file nhưng lại vi phạm quy ước làm việc trực tiếp tại thư mục làm việc `src/` đã được thống nhất trong quy định học phần.

**Student Fix:** Sinh viên đã can thiệp rà soát (Human-in-the-loop), yêu cầu AI sửa lại đường dẫn lưu file trực tiếp tại `src/ai-audit/ai_audit_report.md` và cập nhật lại quy tắc trong `AGENTS.md` để đảm bảo tính gọn nhẹ và chính xác cho thư mục bài nộp.

### 2.2.2 Entry 2

#### Prompt + Tool:

Time: 2026-09-01 00:30 +07 Tool: Gemini 3.6 Flash Prompt: /using-superpowers Hãy cài đặt những công cụ / thư viện... cần thiết phục vụ cho homework này. Viết audit

**AI Output:** Đã rà soát môi trường EShop (backend, frontend-admin, frontend-web đều đã sẵn có node\_modules), liệt kê và cung cấp câu lệnh cài đặt các công cụ kiểm thử hiệu năng/bằng chứng (Java 17 JRE, Apache JMeter, htop, screenfetch, Kazam/OBS) và cập nhật báo cáo AI Audit Report dưới dạng entry 2 trong `src/ai-audit/ai_audit_report.md`.

**Verdict:** Accepted

**Reasoning:** Dựa trên tiêu chí Environmental Readiness & Functional Correctness (ISTQB CT-AI 2.1), AI đã rà soát môi trường Fedora Linux chính xác, xác nhận Node.js backend/frontend đã có sẵn `node_modules` và đề xuất chính xác các công cụ kiểm thử hiệu năng (Apache JMeter, Java 17 JRE) cùng bộ công cụ theo dõi tài nguyên/bằng chứng phần cứng (`htop`, `fastfetch`).

**Student Fix:** Sinh viên trực tiếp thực thi các lệnh cài đặt trên terminal Fedora, kiểm tra phiên bản Java (`java -version`) và JMeter CLI (`jmeter -v`) để đảm bảo các công cụ sẵn sàng trước khi chạy kịch bản kiểm thử.

### 2.2.3 Entry 3

#### Prompt + Tool:

Time: 2026-09-01 16:08 +07 Tool: Gemini 3.6 Flash Prompt: /using-superpowers Dựa vào folder req (chứa đề bài tập) và các spec trong folder eshop-sut (eshop-sut/README.md và eshop-sut/api\_specification.md). Hãy tạo 2 file CSV dữ liệu đầu vào cho luồng Admin (Member 4) tại `src/data/credentials.csv` (chứa tài khoản admin@eshop.com) và `src/data/products.csv` (chứa dữ liệu sản phẩm mẫu). Lưu ý có thể tham khảo mã nguồn của eshop-sut để đảm bảo data tạo ra có thể dùng được. Đưa ghi nhận vào AI Audit Report.

**AI Output:** Đã tham chiếu `eshop-sut/backend/database.js` và `api_specification.md` để lấy chính xác thông tin tài khoản seeded (`admin@eshop.com / Admin123!`) cùng cấu trúc bảng `products` (`name, price, description, imageUrl, category_id`). Đã tạo 2 file CSV tại `src/data/credentials.csv` và `src/data/products.csv` đồng thời ghi nhận Entry 3 vào `src/ai-audit/ai_audit_report.md`.

**Verdict:** Accepted

**Reasoning:** Tiêu chí Data Quality & Alignment (ISTQB CT-AI 2.4 - Sample Bias Prevention). AI đã đọc mã nguồn backend `eshop-sut/backend/database.js` và `api_specification.md` để trích xuất chính xác thông tin tài khoản seeded (`admin@eshop.com / Admin123!`) cùng cấu trúc bảng `products`. Điều này đảm bảo dữ liệu test đầu vào hoàn toàn hợp lệ với cơ sở dữ liệu SQLite của ứng dụng SUT.

**Student Fix:** Sinh viên mở trực tiếp các file CSV tại `src/data/` để kiểm tra định dạng UTF-8, kiểm tra số lượng dòng/cột và đối chiếu với database SQLite thực tế trước khi cấu hình vào JMeter CSV Data Set Config.

## 2.2.4 Entry 4

### Prompt + Tool:

Time: 2026-09-01 16:14 +07 Tool: Claude Opus 4.6 Prompt: /subagent-driven-development Tôi đã tạo data tại `src/data`. Dựa vào yêu cầu trong folder req, hãy thiết kế kịch bản Load Test Plan cho luồng Admin của sinh viên 23127075 (10 threads, ramp-up 10s, dùng Aggregate Report) cho project eshop-sut và lưu tại `src/test-plans/23127075_Load_20260901.jmx`. Bạn có thể sửa test data nếu chưa phù hợp. Đưa ghi nhận vào AI Audit Report.

**AI Output:** Đã đọc mã nguồn backend `eshop-sut/backend/server.js` và `database.js` để xác minh chính xác các API endpoint, middleware xác thực, và cấu trúc dữ liệu. Phát hiện rằng `POST /api/products` và `DELETE /api/products/:id` không yêu cầu `authenticateToken` (khác với `GET /api/admin/users` có yêu cầu). Đã tạo file `src/test-plans/23127075_Load_20260901.jmx` gồm 6 HTTP Samplers theo luồng Admin end-to-end: (1) `POST /api/login` với JWT extraction, (2) `GET /api/admin/users` với Bearer Auth, (3) `GET /api/products`, (4) `GET /api/categories`, (5) `POST /api/products` tạo sản phẩm test từ CSV, (6) `DELETE /api/products/${created_product_id}` dọn dẹp. Cấu hình: 10 threads, ramp-up 10s, 5 loops, Gaussian Random Timer 1-3s, Aggregate Report listener xuất raw.jtl. Mỗi sampler có Response Assertion kiểm tra HTTP 200.

**Verdict:** Passed with Modification

**Reasoning:** Dựa trên tiêu chí Explainability & Functional Correctness (ISTQB CT-AI 2.7). AI thiết kế kịch bản XML chuẩn JMeter cho 10 threads, 10s ramp-up, bao phủ đủ 6 HTTP Samplers end-to-end (Login, Admin Users, Products, Categories, Create Product, Delete Product). AI giải thích và trích xuất thành công JWT token bằng JSON Extractor. Tuy nhiên, AI thiết lập đường dẫn file CSV dạng tuyệt đối (`/home/tkin/...`), gây lỗi khi đem thư mục nộp bài sang môi trường máy khác.

**Student Fix:** Sinh viên yêu cầu rà soát lại và sửa Gaussian Random Timer thành mean 2.000 ms, độ lệch chuẩn 333 ms (xấp xỉ 99,7% trong 1-3 giây, không phải khoảng chặn cứng), giữ ID động trong request path nhưng bỏ khỏi sampler label, rồi chạy lại sạch 300 samples và đối chiếu JTL với HTML report.

## 2.2.5 Entry 5

**Prompt + Tool:**

Time: 2026-09-01 16:19 +07 Tool: Claude Opus 4.6 Prompt: /using-superpowers Tôi đã tạo data tại src/data. Dựa vào yêu cầu trong folder req, hãy tạo kịch bản Stress Test Plan cho luồng Admin (50 threads, ramp-up 15s, dùng Summary Report listener) cho project eshop-sut và lưu tại src/test-plans/23127075\_Stress\_20260901.jmx. Bạn có thể sửa test data nếu chưa phù hợp. Đưa ghi nhận vào AI Audit Report.

**AI Output:** Tái sử dụng cùng luồng 6-sampler Admin (Login → Admin Users → Products → Categories → Create Product → Delete Product) từ Load Test nhưng với cấu hình stress: 50 threads (gấp 5 lần Load), ramp-up 15s, 10 loops = 500 iterations tổng. Giảm think-time xuống 0.5-1.5s (đồng thời ramp nhanh hơn) để tạo áp lực lên Node.js + SQLite backend. Listener đổi sang **Summary Report** (`guiclass="SummaryReport"`) theo yêu cầu. Output raw.jtl tại `src/results/stress/raw.jtl`. Tất cả assertions và JWT extraction giữ nguyên.

**Verdict:** Passed with Modification

**Reasoning:** Dựa trên tiêu chí Flexibility & Boundary Testing (ISTQB CT-AI 2.1). AI cấu hình đúng kịch bản Stress Test với 50 threads (gấp 5 lần Load), ramp-up 15s, 10 loops (500 iterations) và think-time 0.5-1.5s để tạo áp lực liên tục nhằm tìm điểm gãy (breaking point) của hệ thống. Listener được đổi sang **Summary Report** theo đúng yêu cầu. Tuy nhiên, AI chưa lường trước trường hợp 50 vusers login liên tục có thể kích hoạt cơ chế khóa tài khoản (Account Lockout) nếu credentials lỗi.

**Student Fix:** Sinh viên yêu cầu sửa Gaussian Random Timer thành mean 1.000 ms, độ lệch chuẩn 167 ms (xấp xỉ 99,7% trong 0,5-1,5 giây), ổn định sampler label và chạy lại 3.000 samples. Kết quả mới được kiểm tra 0 lỗi và khớp tổng mẫu giữa raw JTL với HTML report.

## 2.2.6 Entry 6

**Prompt + Tool:**

Time: 2026-09-01 16:24 +07 Tool: Claude Opus 4.6 Prompt: /using-superpowers Tôi đã tạo data tại src/data. Dựa vào yêu cầu trong folder req, hãy tạo kịch bản Spike Test Plan cho luồng Admin (100 threads, ramp-up 1s, dùng View Results Tree listener) cho project eshop-sut và lưu tại src/test-plans/23127075\_Spike\_20260901.jmx. Bạn có thể sửa test data nếu chưa phù hợp. Đưa ghi nhận vào AI Audit Report.

**AI Output:** Tạo kịch bản Spike Test với cấu hình cực đoan: 100 threads, ramp-up chỉ 1s (gần như tất cả threads khởi động cùng lúc), 3 loops = 300 iterations. **Không có think-time timer** để tối đa hóa độ sốc đồng thời lên backend. Listener dùng **View Results Tree** (`guiclass="ViewResultsFullVisualizer"`) với `responseData=true`, `samplerData=true`, `responseHeaders=true`, `requestHeaders=true`, `responseDataOnError=true` để lưu chi tiết request/response cho từng sample — hữu ích để debug các lỗi dưới áp lực spike. Cùng luồng 6-sampler Admin giữ nguyên. Output tại `src/results/spike/raw.jtl`.

**Verdict:** Passed with Modification

**Reasoning:** Dựa trên tiêu chí Robustness under Extreme Operational Environment (ISTQB CT-AI 2.1). AI thiết lập tải đột biến cực đoan với 100 threads, ramp-up 1s và 0s think-time, tạo ra độ sốc dồn dập 338.2 RPS lên backend. Listener **View Results Tree** được bật đầy đủ tùy chọn lưu header/body để phục vụ debug chuyên sâu khi có lỗi xảy ra.

**Student Fix:** Sinh viên phát hiện `raw.jtl` đã bị nối từ tám lần chạy thành 14.400 samples trong khi HTML chỉ chứa 1.800 samples. Sinh viên yêu cầu dọn đồng thời JTL/HTML, bỏ ID động khỏi sampler label và chạy lại; artifact cuối có 1.800 samples, 0 lỗi, khớp HTML report.

## 2.2.7 Entry 7

### Prompt + Tool:

Time: 2026-09-01 19:28 +07 Tool: Gemini 3.6 Flash Prompt: /using-superpowers Do thư mục src sẽ được copy ra thành thư mục nộp độc lập, hãy tạo script tự động chạy kiểm thử tại src/run\_tests.sh và cập nhật lại toàn bộ đường dẫn tương đối trong các kịch bản JMeter (.jmx) cho phù hợp với thư mục thực thi src. Đưa ghi nhận vào AI Audit Report.

**AI Output:** Đã tạo script `src/run_tests.sh` tự động thiết lập working directory về chính thư mục `src/`, tự động khởi tạo các thư mục kết quả (`results/load`, `results/stress`, `results/spike`, `evidence/screenshots`, `evidence/hardware`) và dọn dẹp log cũ trước khi thực thi. Đã cập nhật tất cả 3 file kịch bản .jmx (`23127075_Load_20260901.jmx`, `23127075_Stress_20260901.jmx`, `23127075_Spike_20260901.jmx`), đổi toàn bộ đường dẫn CSV DataSet từ `src/data/...` sang `data/...` và Listener output từ `src/results/...` sang `results/...` để đảm bảo khi copy folder `src` thành bài nộp độc lập (VD: `23127075_HW05_AI_Performance_100/`) thì mọi đường dẫn thực thi đều chính xác 100%.

**Verdict:** Passed with Modification

**Reasoning:** Dựa trên tiêu chí Portability & Automation (ISTQB CT-AI 2.1). AI đã tạo script `src/run_tests.sh` tự động định vị làm việc tại `src/`, khởi tạo cây thư mục kết quả (`results/`, `evidence/`) và dọn dẹp log cũ. AI cập nhật thành công toàn bộ đường dẫn CSV và Listener output trong 3 file .jmx từ đường dẫn tuyệt đối sang tương đối (`data/`, `results/`), giúp bộ bài làm hoàn toàn độc lập và sẵn sàng để đóng gói zip.

**Student Fix:** Sinh viên yêu cầu runner dùng `JMETER_BIN` có mặc định `/opt/jmeter/bin/jmeter`, dọn cả raw JTL lẫn HTML trước mỗi lượt và chạy lại trọn bộ. Validator tự động kiểm tra số mẫu dự kiến, lỗi, nhãn ổn định và tính nhất quán JTL/HTML.

## 2.2.8 Entry 8

### Prompt + Tool:

Time: 2026-09-01 20:17 +07 Tool: Gemini 3.6 Flash Prompt: /using-superpowers Hãy convert .agents/skills/ai-audit-entry/ref/13\_Performance Testing.pdf ra markdown đầy đủ nội dung, layout (cố gắng convert full cả table, diagram thay vì cap ảnh)

**AI Output:** Đã thực hiện OCR và đọc toàn bộ 30 trang slide của file tài liệu `13_Performance Testing.pdf` (ThS. Trần Duy Hoàng, FIT @ HCMUS). Đã trích xuất và số hóa toàn bộ nội dung sang file Markdown tại `.agents/skills/ai-audit-entry/ref/13_Performance Testing.md`. Chuyển đổi đầy đủ bảng 10 chỉ số kiểm thử hiệu năng (Resource, Time, Volume Metrics), sơ đồ tác động 1s độ trễ kinh doanh, sơ đồ 6 loại Performance Testing và sơ đồ quy trình 7 bước thực hiện kiểm thử thành các **Mermaid Flowchart** chuẩn HTML/Markdown.

**Verdict:** Passed with Modification

**Reasoning:** Dựa trên tiêu chí Transparency & Interpretability (ISTQB CT-AI 2.7). AI đã OCR và số hóa 30 trang slide tài liệu kiểm thử hiệu năng từ PDF sang Markdown chất lượng cao với đầy đủ bảng biểu. Tuy nhiên, AI đã gặp lỗi cú pháp khi vẽ biểu đồ Mermaid ở slide chỉ số hiệu năng (dùng cú pháp `grid` không nằm trong chuẩn hiển thị Mermaid parser).

**Student Fix:** Sinh viên phát hiện lỗi syntax rendering của Mermaid, thực hiện chỉnh sửa sơ đồ từ `grid` thành `flowchart LR` kết hợp các `subgraph` giúp biểu đồ 10 chỉ số hiệu năng hiển thị chuẩn xác và trực quan.

### 2.2.9 Entry 9

#### Prompt + Tool:

Time: 2026-09-01 21:23 +07 Tool: Claude Opus 4.6 Prompt: /using-superpowers /writing-skills Hãy viết agent skill: performance-testing-skills để automate hoàn thành bài tập theo đề trong folder req với các yêu cầu sau:

- Tham khảo lý thuyết từ slide (file markdown) trong folder ref.
- Tham khảo src/ai-audit/ai\_audit\_report.md để hiểu quá trình làm bài.
- Bộ skill phải đầy đủ bước cho cả workflow và đảm bảo tính độc lập, tái sử dụng, portable, không lệ thuộc vào các resource (như kiểu phải đưa slide vào để học kiến thức).
- Output skill nằm trong .agents/skills Hãy viết 1 file hướng dẫn sử dụng bộ skill này ở root. Ghi prompt này vào ai-audit.

#### AI Output:

```
Đã đọc và tuân thủ skill `writing-skills` và `using-superpowers`. Tham khảo đầy đủ lý thuyết từ `.agents/skills/ai-audit-entry/ref/13_Performance Testing.md` (30 slide), đề bài `req/2026.HW05.Performance Testing_Vi.md`, và quá trình làm bài trong `src/ai-audit/ai_audit_report.md` (Entry 1-8). Đã tạo bộ skill `performance-testing-skills` tại `.agents/skills/performance-testing-skills/` gồm:
```

- `SKILL.md` (11KB): Skill chính với 10-phase workflow đầy đủ, lý thuyết kiểm thử hiệu năng đã tích hợp sẵn (10 metrics, 6 test types, 7-step process), JMeter CLI reference, deliverables checklist, common mistakes table.
- `references/jmeter-plan-template.md` (5.8KB): Template XML cho ThreadGroup, CSV DataSet, JSON Extractor, Assertion, Timer, Listener.
- `references/report-templates.md` (4.9KB): Template báo cáo chính, README, AI Critique.
- `scripts/run\_tests\_template.sh` (2.4KB): Template script chạy test với biến cấu hình.

```
Đồng thời tạo file hướng dẫn sử dụng `GUIDE_PERFORMANCE_TESTING_SKILLS.md` tại root với bảng 10 phase, prompt mẫu cho từng phase, và lưu ý quan trọng.
```

**Verdict:** Passed with Modification

**Reasoning:** Dựa trên tiêu chí Autonomous Agent Synthesis & Quality Verification (ISTQB CT-AI 2.2 & 2.7). AI tuân thủ nghiêm ngặt quy trình `writing-skills`, tự tổng hợp toàn bộ lý thuyết kiểm thử hiệu năng (10 metrics, 6 loại test, 7 bước) và 10-phase workflow vào bộ skill `performance-testing-skills` tại



`.agents/skills/performance-testing-skills/`. Skill đảm bảo tính tự chứa (self-contained), có sẵn các template XML, Markdown report và script mẫu.

**Student Fix:** Sinh viên yêu cầu sửa template assertion sang `Assertion.response_code`, mô tả đúng Gaussian theo mean/standard deviation, thêm analyzer JTL, monitor CPU theo interval, runner endurance, kiểm tra hồi quy cho skill và quy tắc copy skill ngoài `src/` vào staging khi đóng gói.

### 2.2.10 Entry 10

#### Prompt + Tool:

Time: 2026-09-03 00:24 +07 Tool: Codex / GPT-5 Prompt: /using-superpowers /performance-testing-skills Hãy rà soát bài làm so với đề, sửa JMX để kết quả có thể tổng hợp ổn định, tạo Endurance 10 phút có log CPU/RSS thật, thêm công cụ phân tích JTL và validator đối chiếu JTL/HTML, sau đó cập nhật báo cáo bằng số liệu đã chạy. Không tạo PDF, video hoặc bằng chứng GitHub giả.

**AI Output:** Đã sửa Gaussian timer và sampler label, chạy lại sạch Load/Stress/Spike, tạo và chạy Endurance 10 phút, thu thập raw JTL/HTML/CPU-RSS, thêm analyzer và validator có regression tests, cập nhật báo cáo và bug report the...

**Verdict:** Passed with Modification

**Reasoning:** AI đã tạo được artifact mới và chạy kiểm chứng end-to-end: Endurance đủ 10 phút có raw JTL, HTML và 121 mẫu CPU/RSS; analyzer phân biệt `elapsed` với `Latency`; validator phát hiện sample mismatch và bốn blocker thủ công. Tuy nhiên, AI không thể tự hoàn thành video có giọng sinh viên, PDF theo lựa chọn của sinh viên, hay screenshot GitHub Issue khi token không hợp lệ.

**Student Fix:** Sinh viên yêu cầu giữ nguyên bằng chứng chạy thật, không bịa video/PDF/GitHub screenshot, hạ self-assessment khỏi 100 và chỉ coi bài sẵn sàng nộp sau khi validator hết bốn blocker thủ công. Sau review, sinh viên yêu cầu dọn năm sản phẩm test bị scheduler bỏ giữa iteration, ghi cleanup evidence, kiểm tra resource-log coverage/assertion/package naming và đổi “ngưỡng tối đa” thành “điểm tải bền vững đã chứng minh”.

## 3. Tổng kết độ chính xác AI

- **Tỷ lệ chính xác ước tính:** Không gán phần trăm chủ quan; mỗi artifact được đánh giá theo verdict và bằng chứng riêng.
- **Điểm mạnh của AI:** Khả năng tạo nhanh cấu hình kịch bản JMeter XML (`.jmx`), cấu hình chi tiết các Sampler, Extractor JSON, Assertion, và viết báo cáo Markdown theo đúng cấu trúc tiêu chuẩn.
- **Điểm yếu của AI:** Đã mô tả sai Gaussian timer, đặt biến động trong sampler label, không phát hiện JTL bị nối khác tổng mẫu HTML, nhầm average với p95 và gọi throughput trung bình là giá trị đỉnh. Một số đề xuất như Redis cluster không phù hợp với SQLite đơn tiến trình.

## 4. Kết luận

AI giúp tăng tốc việc sinh plan, script và khung báo cáo, nhưng chất lượng cuối phụ thuộc vào human review. Số liệu trong báo cáo đã được tính lại từ raw JTL, đối chiếu với HTML report và tách riêng các blocker chưa có bằng chứng thay vì tuyên bố hoàn thành tuyệt đối.

## 5. Disclosure

Tôi có sử dụng các công cụ AI (Gemini 3.6 Flash, Claude Opus 4.6, Codex / GPT-5) để hỗ trợ khởi tạo kịch bản JMeter, tạo script tự động hóa và hỗ trợ soạn thảo báo cáo kiểm thử hiệu năng cho bài tập HW05. Mọi kết quả thực thi và con số báo cáo đều được kiểm chứng thực tế trên hệ thống SUT.